CSC201 Data Structures and Algorithms: Lab Companion Manual

2026/27 Semester 1. Covers arrays, linked lists, stacks, queues, recursion, searching and sorting. Lab deadline: 28 February 2027.

How to Use This Manual

This is a guide, not a workbook. There are no blanks to fill in. Each chapter matches one lab task and the pages of the lab manual PDF where that task lives. Read the PDF pages first, then read the matching chapter here, then write and run the code yourself.

Every chapter follows the same pattern:

1. **Goal**: what the task asks for.
2. **Core idea**: the concept in plain language.
3. **Build it step by step**: how to construct the solution yourself.
4. **Trace it**: walk through the sample run by hand, which is how you prove you understand it.
5. **Pitfalls and self-check**: common mistakes and questions to test yourself.

Study method that works: type the code out rather than pasting it, predict the output before running, and when the output differs from your prediction, find out why before moving on.

Page Map

Task	Topic	PDF pages	Points
1.1	Dynamic array	6-8	10
1.2	Reverse array in place	9-11	10
2.1	Linked list insertion	12-14	10
2.2	Linked list deletion and traversal	15-17	15
3.1	Stack	24-26	10
3.3	Queue	30-32	10

Task	Topic	PDF pages	Points
3.5	Balanced parentheses	36-38	15
4.1	Factorial and Fibonacci	42-44	10
5.2	Binary search	50-52	10
5.6	Merge sort	59-62	20

Pages not listed (for example 18-23, 27-29, 33-35, 39-41, 45-49, 53-58) are supporting theory and exercises in the PDF that have no graded task. Read them anyway; they build the background for the tasks.

Setup

You need Python 3 and any editor. Save each task as its own file such as `task_1_1.py` and run it with `python task_1_1.py`. Compare your output with the expected output given in each chapter.

Part 1: Arrays

Task 1.1: Dynamic Array Implementation

PDF pages 6-8 | 10 points

Goal

Build a `DynamicArray` class with `append(value)` and `get(index)` that grows automatically when full.

Core idea

A normal array has a fixed size. A dynamic array hides that limit. It keeps a hidden fixed-size storage list and tracks two numbers:

- **capacity**: how many slots exist
- **size**: how many slots are actually used

When `size == capacity`, there is no room. The array allocates new storage **twice as large**, copies every element across, and carries on. Doubling matters: it makes appending **amortised O(1)**, meaning that although one resize costs O(n), resizes are rare enough that the average cost per append stays constant.

Build it step by step

1. In `__init__`, set `capacity = 2`, `size = 0`, and `array = [None] * capacity`.
2. In `append`, first check `if self.size == self.capacity`. If true: double the capacity, create a new list of the new capacity, copy the old elements with a loop, and replace `self.array`.
3. Place the value at index `self.size`, then increase `size` by 1. The order matters: write first, increment second.
4. In `get`, check bounds against **size**, not capacity. Slots between size and capacity hold `None` and are not real data. Raise `IndexError` if the index is out of range.

Trace it

Start: capacity 2, size 0.

- append(10): room, size 1.
- append(20): room, size 2. Now full.
- append(30): full, so capacity becomes 4, copy 10 and 20, place 30, size 3.

Printed: `10`, `30`, `4`, `3`.

Pitfalls and self-check

- Checking bounds against capacity lets users read empty slots.
- Forgetting to update `self.array` after copying means the new storage is thrown away.
- Ask yourself: what would capacity be after 9 appends? (Answer: 16.) Why is doubling better than adding 1 slot each time?

Task 1.2: Reverse an Array In-Place

PDF pages 9-11 | 10 points

Goal

Write `reverse_array(arr)` that reverses a list in place, without `reverse()` or slicing like `arr[::-1]`.

Core idea

"In place" means no second list. Use the **two-pointer technique**: one pointer at the start, one at the end. Swap their elements, then move them towards each other until they meet.

Build it step by step

1. Set `left = 0` and `right = len(arr) - 1`.
2. Loop while `left < right`. Using `<` rather than `<=` matters because when the pointers meet on the middle element of an odd-length list, there is nothing to swap.
3. Swap with Python's tuple assignment: `arr[left], arr[right] = arr[right], arr[left]`.
4. Move inward: `left += 1`, `right -= 1`.
5. Return the list.

Trace it

`[1, 2, 3, 4, 5]`: swap positions 0 and 4 giving `[5,2,3,4,1]`; swap 1 and 3 giving `[5,4,3,2,1]`; now left=2, right=2, so stop.

`[7, 2]` becomes `[2, 7]` after a single swap.

Expected output: `[5, 4, 3, 2, 1]` and `[2, 7]`.

Pitfalls and self-check

- Looping the full length swaps everything twice and restores the original order.
- Time is O(n); extra space is O(1). Be able to explain why.
- Test an empty list and a one-element list. Both should return unchanged.

Part 2: Linked Lists

In a linked list, elements are not stored side by side. Each element is a **node** holding a value and a reference (`next`) to the following node. The list itself only remembers the **head**. Anything you want to reach, you reach by walking from the head.

Task 2.1: Singly Linked List, Node and Insertion

PDF pages 12-14 | 10 points

Goal

Create a `Node` class and a `SinglyLinkedList` with `insert_at_head` and `insert_at_tail`.

Core idea

Inserting at the head is O(1): the new node simply points to the old head and becomes the new head. Inserting at the tail is O(n) because you must walk to the last node (the one whose `next` is `None`) before attaching.

Build it step by step

1. `Node.__init__` stores `value` and sets `next = None`.
2. `SinglyLinkedList.__init__` sets `head = None` (an empty list).
3. **insert_at_head**: create the node, set `new_node.next = self.head`, then `self.head = new_node`. Do these in that order, or you lose the rest of the list.
4. **insert_at_tail**: create the node. If the list is empty, make it the head and stop. Otherwise start at head, loop `while current.next is not None`, then set `current.next = new_node`.

Trace it

- insert_at_tail(1): empty, so head is 1.
- insert_at_tail(2): walk to node 1, attach 2. List: 1 -> 2.
- insert_at_head(0): new node points to 1, becomes head. List: 0 -> 1 -> 2.

Expected output: `0 -> 1 -> 2 -> None`.

Pitfalls and self-check

- Assigning `self.head = new_node` before linking `new_node.next` orphans every existing node.
- Forgetting the empty-list case crashes with an `AttributeError` on `None`.
- Draw boxes and arrows for each step. If you can draw it, you understand it.

Task 2.2: Singly Linked List, Deletion and Traversal

PDF pages 15-17 | 15 points

Goal

Add `delete_value(value)` and `traverse()` to the list from Task 2.1.

Core idea

To delete a node, you do not destroy it; you **bypass** it. If node A points to B and B points to C, set A's `next` to C. Nothing refers to B anymore, so it is discarded. This is why you need to stand on the node **before** the one to delete.

Build it step by step

`delete_value` has three cases:

1. **Empty list**: return immediately.
2. **Target is the head**: move the head forward with `self.head = self.head.next`, then return.
3. **Target is further along**: set `current = self.head`. While `current.next` exists, check `current.next.value`. If it matches, do `current.next = current.next.next` and return. Otherwise advance `current`.

`traverse` walks from head to the end, collecting each value as a string, appends `"None"`, and prints them joined with `" -> "`.

Trace it

Build 0 -> 1 -> 2, then delete 1. Standing on node 0, `current.next.value` is 1, a match, so node 0 now points at node 2.

Expected output: `0 -> 2 -> None`.

Pitfalls and self-check

- Checking `current.value` instead of `current.next.value` means you have passed the node and cannot unlink it.
- Forgetting the head case means deleting the first item never works.
- Deleting a value that does not exist should do nothing, with no crash. Test it.

- Only the first match is removed here. How would you change the code to remove all matches?

Part 3: Stacks and Queues

Pages 18-23 (theory leading into stacks) and 27-29 (stack applications) are untasked but worth reading before the tasks below.

Task 3.1: Stack, Array-Based Implementation

PDF pages 24-26 | 10 points

Goal

Implement a `Stack` with `push`, `pop`, `peek` and `is_empty`.

Core idea

A stack is **LIFO**: last in, first out, like a pile of plates. You only ever touch the top. A Python list is ideal because `append()` and `pop()` both work on the end in O(1).

Build it step by step

1. Store a list in `self.items`.
2. `push(item)`: `self.items.append(item)`.
3. `pop()`: remove and return the last item with `self.items.pop()`.
4. `peek()`: return `self.items[-1]` without removing it.
5. `is_empty()`: return whether the length is 0.

Trace it

Push 1, 2, 3 gives `[1, 2, 3]` with 3 on top. `peek()` shows 3 and changes nothing. `pop()` returns 3 and the list becomes `[1, 2]`. The stack is not empty.

Expected output: `3`, `3`, `False`.

Pitfalls and self-check

- `peek` and `pop` differ: one looks, one removes.

- Popping or peeking an empty stack raises `IndexError`. As an improvement, check `is_empty()` first and raise a clearer message.
- Why is the *end* of the list the top, rather than the front?

Task 3.3: Queue, Array-Based Implementation

PDF pages 30-32 | 10 points

Goal

Implement a `Queue` with `enqueue`, `dequeue` and `front`.

Core idea

A queue is **FIFO**: first in, first out, like a line at a bank. Items join at the back and leave from the front.

Build it step by step

1. `enqueue(item)`: `append` to the end of the list.
2. `dequeue()`: remove from the front with `self.items.pop(0)`.
3. `front()`: return `self.items[0]` without removing.

Trace it

Enqueue 1, 2, 3 gives `[1, 2, 3]`. `front()` returns 1. `dequeue()` returns 1, leaving `[2, 3]`. The next `dequeue()` returns 2.

Expected output: `1`, `1`, `2`.

Pitfalls and self-check

- `pop(0)` is O(n) because every remaining element shifts left. For large queues, `collections.deque` with `popleft()` is O(1). Be ready to explain this trade-off.
- Compare with the stack: which end does each structure remove from?

Task 3.5: Balanced Parentheses Checker

PDF pages 36-38 | 15 points

Goal

Write `is_balanced(expression)` that returns `True` when every `()`, `{}` and `[]` is properly matched and nested.

Core idea

The most recent unclosed opener must be the first one closed. That is exactly LIFO behaviour, so a stack is the natural tool.

Build it step by step

1. Create an empty stack and a dictionary mapping each closer to its opener: `{')': '(', ']': '[', '}': '{'}`.
2. Scan the string character by character.
3. If the character is an opener, push it.
4. If it is a closer, there are two failure checks. If the stack is empty, there is nothing to match, so return `False`. Otherwise pop the top and compare it with the expected opener; a mismatch returns `False`.
5. Ignore all other characters.
6. At the end, return `True` only if the stack is empty. Leftover openers mean something was never closed.

Trace it

`{[()]}`: push `{`, `[`, `(`. Then `)` pops `(` correct, `]` pops `[` correct, `}` pops `{` correct. Stack empty, so `True`.

`{[(])}`: push `{`, `[`, `(`. Then `]` pops `(`, which is not `[`, so `False`.

`((`: both pushed, never closed, stack not empty, so `False`.

Expected output: `True`, `False`, `False`.

Pitfalls and self-check

- The order `not stack or stack.pop() != ...` matters: it checks emptiness before popping, avoiding a crash.
- Forgetting the final emptiness check wrongly accepts `((`.
- Test `)(`, an empty string, and `a(b)c`. What should each return?

Part 4: Recursion

Task 4.1: Factorial and Fibonacci

PDF pages 42-44 | 10 points

Goal

Implement `factorial(n)` and `fibonacci(n)` recursively with correct base cases.

Core idea

A recursive function solves a problem by solving a smaller copy of itself. Every recursive function needs:

- a **base case** that stops the recursion without calling itself, and
- a **recursive case** that moves closer to the base case.

Without a base case, the function calls itself forever until Python raises `RecursionError`.

Build it step by step

factorial: n! is n times (n-1)!, and 0! and 1! are both 1.

1. Base case: `if n <= 1: return 1`.
2. Recursive case: `return n * factorial(n - 1)`.

fibonacci: each number is the sum of the previous two, starting 0, 1, 1, 2, 3, 5, 8.

1. Base case: `if n <= 1: return n` (so fib(0) = 0 and fib(1) = 1).
2. Recursive case: `return fibonacci(n - 1) + fibonacci(n - 2)`.

Trace it

`factorial(5)` expands to 5 * 4 * 3 * 2 * 1 = 120. Write out the call stack: each call waits for the one below it, then the answers multiply back up as the calls return.

`fibonacci(6)`: the sequence 0, 1, 1, 2, 3, 5, 8 puts the answer at position 6, which is 8.

Expected output: `120`, `8`.

Pitfalls and self-check

- Fibonacci needs **two** base cases hidden in `n <= 1`; this version makes two calls per level, so it is O(2^n). Draw the call tree for `fibonacci(5)` and count repeated work such as `fibonacci(2)` computed several times.
- Research **memoization** (caching results) as the fix, which brings it down to O(n).
- What happens if you call `factorial(-3)`, and what would you add to handle it properly?

Part 5: Searching and Sorting

Pages 45-49 (searching theory) and 53-58 (sorting theory) are untasked background.

Task 5.2: Binary Search

PDF pages 50-52 | 10 points

Goal

Implement both an iterative and a recursive binary search on a **sorted** array. Return the index of the target, or -1.

Core idea

Look at the middle element. If it is the target, done. If the target is larger, it can only be in the right half; if smaller, only in the left half. Each step discards half of what remains, giving O(log n). One million items need at most about 20 comparisons.

The array **must** be sorted, or the halving logic is invalid.

Build it step by step: iterative

1. Set `low = 0`, `high = len(arr) - 1`.
2. While `low <= high`: compute `mid = (low + high) // 2`.
3. If `arr[mid] == target`, return `mid`.
4. If `arr[mid] < target`, set `low = mid + 1`; otherwise set `high = mid - 1`.
5. If the loop ends, return -1.

Build it step by step: recursive

The same logic, with `low` and `high` as parameters. Base cases: `low > high` returns -1, and a match returns `mid`. Otherwise call itself on the right half (`mid + 1, high`) or left half (`low, mid - 1`). Use `high=None` as the default and set it to `len(arr) - 1` on the first call.

Trace it

Search 7 in `[1, 3, 5, 7, 9, 11]`: low 0, high 5, mid 2 holds 5, which is less than 7, so low becomes 3. Then mid 4 holds 9, which is greater than 7, so high becomes 3. Then mid 3 holds 7. Found, index 3.

Search 4: it narrows down until `low > high` with no match, so -1.

Expected output: `3`, `-1`.

Pitfalls and self-check

- Using `mid` instead of `mid + 1` or `mid - 1` can cause an infinite loop.
- Using `<` instead of `<=` in the loop skips the final remaining element.
- Which version uses more memory, and why? (Recursion uses call-stack space, O(log n); iteration uses O(1).)

Task 5.6: Merge Sort

PDF pages 59-62 | 20 points

This is the highest-weighted task. Give it the most time.

Goal

Implement `merge_sort(arr)` using divide and conquer, with a helper `merge(left, right)`.

Core idea

Three phases:

1. **Divide**: split the list in half.
2. **Conquer**: sort each half recursively.

3. **Combine**: merge the two sorted halves into one sorted list.

A list of 0 or 1 elements is already sorted. That is the base case. Time is O(n log n) in all cases; extra space is O(n).

Build it step by step: `merge`

Merging two **already sorted** lists is the heart of the algorithm.

1. Create an empty `result` and two indexes, `i = j = 0`.
2. While both lists still have unread items, compare `left[i]` and `right[j]`. Append the smaller and advance that index. Using `<=` keeps equal items in original order, which makes the sort **stable**.
3. When one list runs out, the other still has items left over, all larger than anything placed so far. Append them with `extend(left[i:])` and `extend(right[j:])`.
4. Return `result`.

Build it step by step: `merge_sort`

1. Base case: if `len(arr) <= 1`, return `arr`.
2. Find `mid = len(arr) // 2`.
3. Sort `arr[:mid]` and `arr[mid:]` recursively.
4. Return `merge(left, right)`.

Trace it

`[38, 27, 43, 3, 9, 82, 10]` splits into `[38, 27, 43]` and `[3, 9, 82, 10]`, then continues splitting down to single elements. On the way back up, merging `[27, 38]` with `[43]` gives `[27, 38, 43]`, and merging `[3, 9]` with `[10, 82]` gives `[3, 9, 10, 82]`. The final merge of those two gives the result.

Expected output: `[3, 9, 10, 27, 38, 43, 82]`.

Pitfalls and self-check

- Practise `merge` on its own first with `[1, 4, 7]` and `[2, 3, 9]` before testing the full sort.
- Forgetting the leftover `extend` steps silently drops elements.
- Draw the recursion as an upside-down tree. How many levels are there for 8 elements? (Three, hence the log n.)

- Compare with binary search: both halve the problem, but merge sort must also process all n elements at each level.

Final Checklist Before Submission

- Each task is saved in its own file and runs without errors.
- Output matches the expected output exactly, including spacing and line order.
- You can explain every function aloud without reading the code.
- You can state the time complexity of each task: dynamic array append O(1) amortised; reverse O(n); list head insert O(1), tail insert O(n); stack operations O(1); queue with list dequeue O(n); balanced check O(n); factorial O(n); naive Fibonacci O(2^n); binary search O(log n); merge sort O(n log n).
- You have tested edge cases: empty input, one element, duplicates, and missing values.

Suggested schedule: Parts 1 and 2 in the first week, Part 3 in the second, then Part 4 and Part 5 in the third, leaving the remaining time for review well before the deadline.